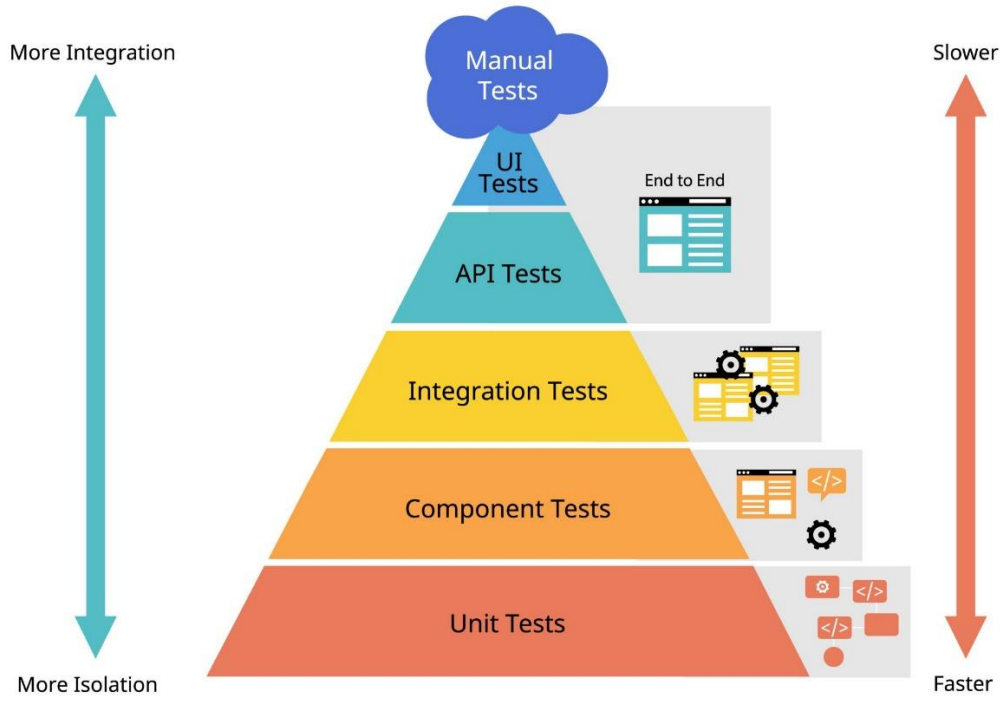
UNIDAD #4: PRUEBAS DEL SISTEMA.

En la fase de Fábrica del modelo PPARD, las pruebas no son un evento de última hora, sino un proceso continuo de control de calidad. Un sistema no probado no es un producto; es una hipótesis arriesgada, como futuros ingenieros, su responsabilidad es demostrar activamente que el sistema falla bajo condiciones controladas antes de que lo haga de forma catastrófica en producción.

Es común que los desarrolladores subestimen o intenten "saltarse" esta etapa porque prefieren la gratificación inmediata de escribir funciones nuevas. Sin embargo, un sistema sin pruebas es una deuda técnica con intereses al 100%. En esta unidad no nos quedaremos en la teoría abstracta de "qué es un error"; el objetivo es desarrollar la mentalidad de romper el código antes de que lo haga el usuario.

4.1. Pirámide de pruebas.



- **Pruebas Unitarias:** Validan la lógica atómica, deben ser cientos, rápidas y ejecutarse en cada cambio de código.
- **Pruebas de componentes:** Verifican que un módulo completo (un conjunto de funciones o un componente de UI en React/TS) funcione correctamente de forma aislada antes de integrarse con el resto.
- **Pruebas de integración:** Comprueban la comunicación entre el código y agentes externos (bases de datos, APIs de terceros o sistemas de archivos). No validan la lógica interna, sino los canales de comunicación y contratos de datos.
- **Pruebas de API:** Envían peticiones HTTP (GET, POST, etc.) a los endpoints del backend para verificar que las respuestas devuelvan el status code y la estructura JSON esperados. Aseguran el "contrato de integración de interfaces" entre el Backend y el Frontend.
- **Pruebas de UI / End To End (E2E):** Simulan el flujo completo de un usuario real (clics, navegación, flujos de compra). Son de alto valor, pero frágiles: un cambio estético en el HTML (como un ID o una clase) puede romper el test, aunque la lógica del sistema siga intacta.
- **Test manuales (Exploratorias):** Ejecutadas por un humano para evaluar usabilidad, accesibilidad y flujos no previstos por las máquinas. Deben ser el porcentaje más pequeño de la suite para evitar cuellos de botella en el despliegue.
- **Regla de oro de la automatización:** Automatiza lo pequeño de forma masiva para que, al llegar a la cima de la pirámide, los cimientos del sistema sean indestructibles.

4.2. Técnicas de Diseño de Pruebas: Caja Negra vs. Caja Blanca.

No probamos disparando a ciegas. Abordamos el sistema desde dos perspectivas complementarias:

- **Pruebas de Caja Blanca (Pruebas Estructurales):** El evaluador tiene acceso total al código fuente, el objetivo es garantizar la cobertura de código (Code Coverage), asegurando que las pruebas

fuercen la ejecución de cada bifurcación lógica (if, else, match, manejo de excepciones).

- **Pruebas de Caja Negra (Pruebas Funcionales):** El evaluador desconoce el código interno; solo interactúa con las entradas y salidas del sistema basándose en las especificaciones y requerimientos. Se aplican técnicas formales como la **Partición de Equivalencia** y el **Análisis de Valores Límite** para no probar infinitos datos aleatorios.

4.3. Pruebas No Funcionales (El mundo real).

Estas pruebas evalúan el comportamiento y las capacidades del sistema bajo condiciones operativas reales o extremas:

- **Pruebas de Carga:** Evalúan el rendimiento del sistema bajo la demanda máxima proyectada (Ej: ¿Cómo responde el "Servicio de Impresiones" si los 100 estudiantes del turno envían documentos en la misma hora?).
- **Pruebas de Estrés:** Fuerzan al sistema más allá de sus límites operativos para identificar su punto de ruptura y evaluar su capacidad de recuperación (Ej: ¿El servidor se cae por completo o rechaza peticiones de manera controlada al recibir 10 veces su carga máxima?).
- **Pruebas de Usabilidad:** Evalúan qué tan intuitiva es la interfaz, si un estudiante necesita un manual para subir un PDF a imprimir, el diseño ha fallado.
- **Pruebas de Seguridad (Básicas):** Validación y sanitización estricta de inputs para prevenir inyecciones SQL, Cross-Site Scripting (XSS) o desbordamientos de memoria (buffer overflow).

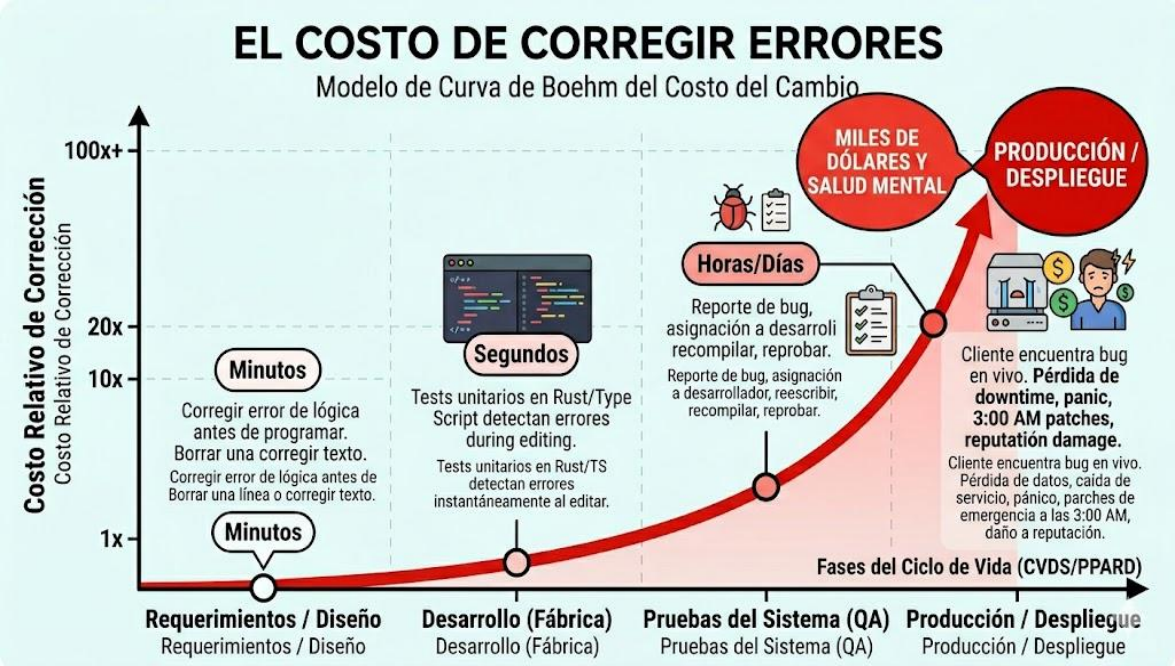
4.4. Gestión de Defectos y el "Bug Report"

Un error mal reportado es un error que no se soluciona. Todo equipo de ingeniería debe adoptar un estándar de reporte para evitar disputas de comunicación ("en mi máquina sí funciona"), un reporte profesional incluye:

- **ID y Título descriptivo:** Directo y claro. *(Ej: "BUG-404: Error 500 al subir archivos .docx mayores a 10MB").*

- **Severidad:** Clasificación del impacto técnico. *(¿Bloquea el flujo principal del sistema o es un detalle visual?)*.
- **Pasos para reproducir:** La secuencia exacta de acciones para recrear el fallo de forma consistente.
- **Resultado Obtenido vs. Resultado Esperado:** Qué hizo el sistema frente a qué debió haber hecho según el requerimiento.
- **Entorno:** Sistema operativo, navegador, versión del compilador o configuración del entorno donde ocurrió.

4.5. Curva de Boehm (Curva del Costo del Cambio)



4.6. Automatización de pruebas.

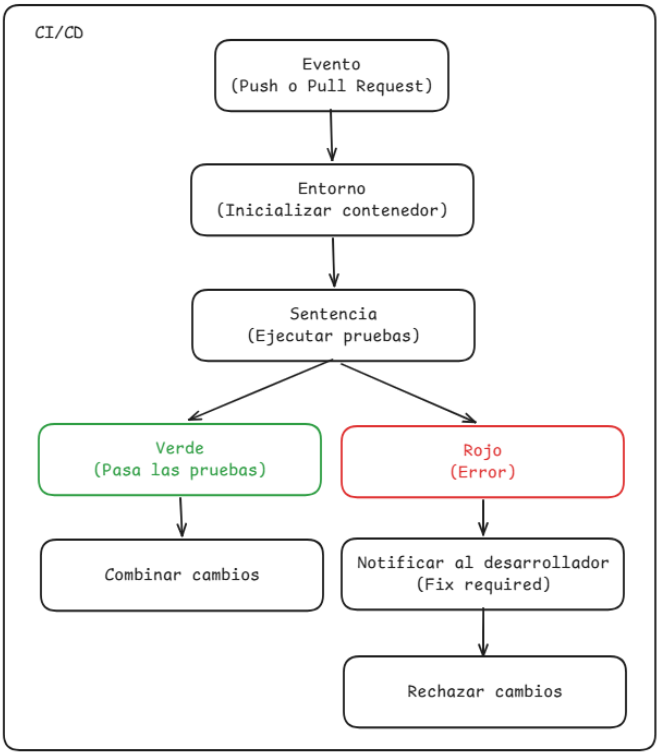
Cuando el código crece a gran velocidad, es humanamente imposible recordar y ejecutar manualmente cada prueba cada vez que hacemos un cambio. Aquí es donde nace la automatización: delegamos la tarea de vigilancia a máquinas que no se cansan, no olvidan y no perdonan errores:

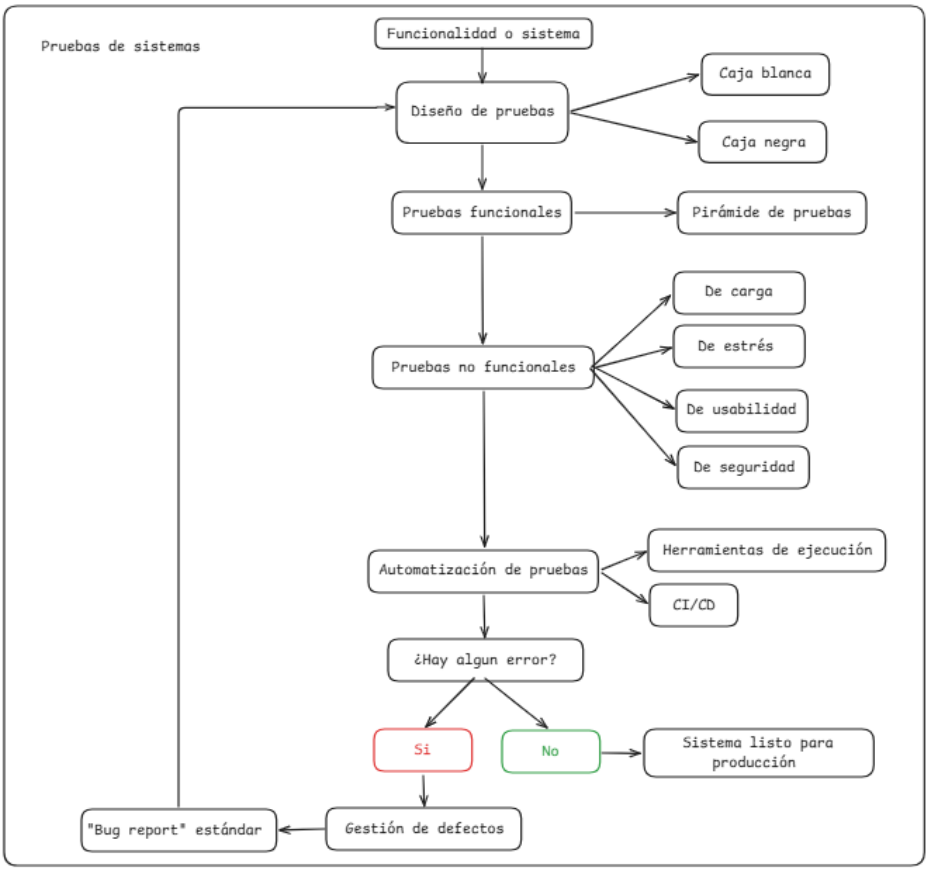
- **Herramientas de Ejecución (Test Runners):** Para dejar de hacer el "trabajo sucio" manualmente, los lenguajes modernos integran herramientas diseñadas para descubrir, ejecutar y reportar el estado de nuestras pruebas.

1. **Rust:** Utiliza su sistema nativo mediante el comando cargo test. Es extremadamente eficiente y viene integrado en el núcleo del lenguaje.
2. **TypeScript:** Se apoya en ecosistemas como Vitest o Jest, que permiten ejecutar pruebas en milisegundos y ofrecen un "modo observación" (watch mode) que reacciona a cada cambio en el código.

- **Integración Continua/Despliegue Continuo (CI/CD):** Es el estándar de la industria bajo la metodología DevOps. El objetivo es que la calidad sea obligatoria, no opcional. A través de herramientas como GitHub Actions, configuramos un flujo de trabajo automatizado mediante un archivo YAML:
    1. **El Evento:** Cada vez que un programador sube código (push) o solicita integrar su trabajo (Pull Request).
    2. **El Entorno:** El sistema levanta automáticamente un servidor virtual (un contenedor) y descarga el código.
    3. **La Sentencia:** El servidor instala las dependencias y ejecuta toda la suite de pruebas.
        - **Si las pruebas pasan (Verde):** El código sigue su camino hacia el despliegue o la integración.
        - **Si una sola prueba falla (Rojo):** El sistema bloquea el proceso. El código no se sube, no se mezcla y no llega a producción.

    El CI/CD es el "filtro de la verdad", si el código no sobrevive a las pruebas automatizadas, simplemente no existe para el resto del equipo, es la única forma de garantizar que una funcionalidad nueva no destruya lo que ya funcionaba ayer.


CI/CD
Evento
(Push o Pull Request)
Entorno
(Inicializar contenedor)
Sentencia
(Ejecutar pruebas)
Verde
(Pasa las pruebas)
Rojo
(Error)
Combinar cambios
Notificar al desarrollador
(Fix required)
Rechazar cambios


Pruebas de sistemas
Funcionalidad o sistema
Diseño de pruebas
Caja blanca
Caja negra
Pruebas funcionales
Pirámide de pruebas
Pruebas no funcionales
De carga
De estrés
De usabilidad
De seguridad
Automatización de pruebas
Herramientas de ejecución
CI/CD
¿Hay algun error?
Si
No
Sistema listo para producción
Gestión de defectos
"Bug report" estándar